

The final assessment for this module consists of 5 Questions. Please read each of the questions carefully and you should attempt **ALL** the questions. Each question is of equal weight. The marking rubric can be seen on the assessment page on Canvas.

Submission Requirements

- Files should be submitted in the .ipynb format.
- Submit either one file per question (5 files total) or one single file containing all answers.

Documentation Requirements

- Comments and docstrings should be concise.
- Comments and docstrings should have a consistent format throughout the solutions.
- Comments should explain the function of the code written.
- Each function must include a **docstring** that explains the purpose of the function, parameters and the return value.
- Variable and function names must be descriptive and follow a consistent naming style, but maximum 25 characters.

Use of Generative AI

You may make use of Generative AI tools when working on this assessments as specified in the **SST policy on the use of AI** (Please see **Appendix 2 of the Programme Handbook**). **All use of Generative AI (GenAI) must be reported** in any assessment submission. This must be done in a way that makes it clear to the reader what is your original work and what has been shaped by GenAI.

1. You are given the following uncommented code for function `estimate_cost`:

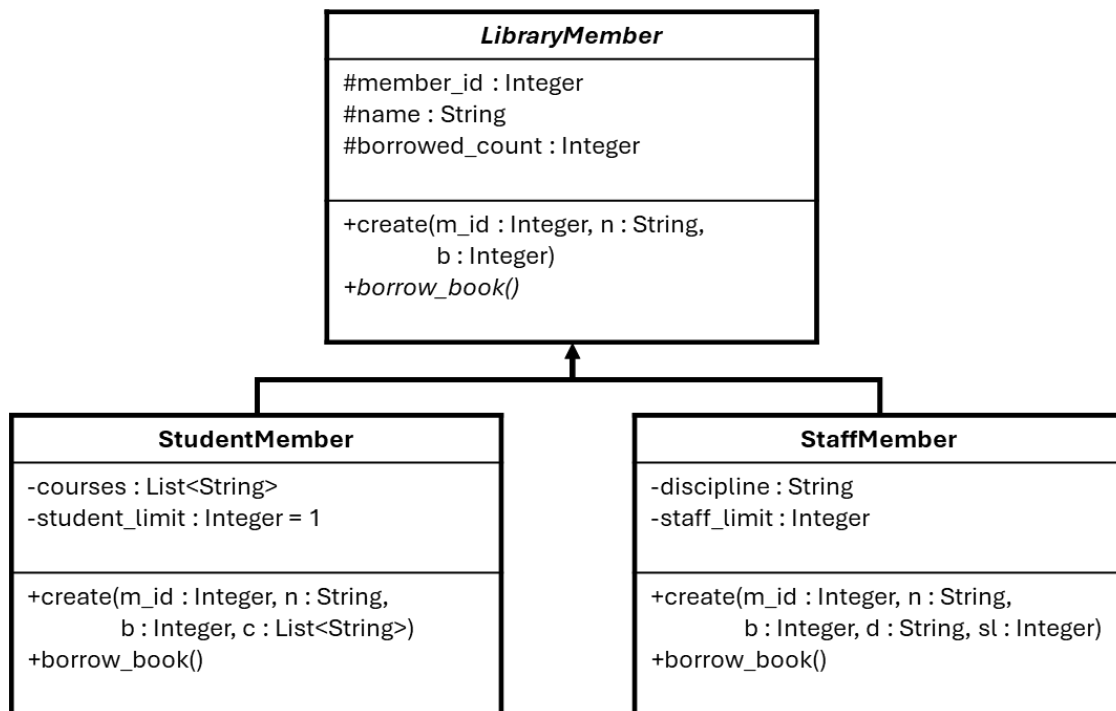
```
def estimate_cost(name, nights, price, breakfast):
    total = nights * price_per_night
    if breakfast:
        total += nights * 12
    print(guest_name, "total cost:", total)

guest_name = "Olivia"
price_per_night = 65

estimate_cost(guest_name, 3, price_per_night, True)
```

- i) Write a new version of the code that has a class called "HotelRoom". This class should have the data as instance variables and a method that has the same functionality as function `estimate_cost`. Then write code to instantiate an object of the `HotelRoom` class.
- ii) Using a docstring, explain in your own words at the start of the class what its purpose is. It should then have clear in-code comments. Docstring and in-code comments should follow the Documentation Requirements.

2. You are given the following class diagram:



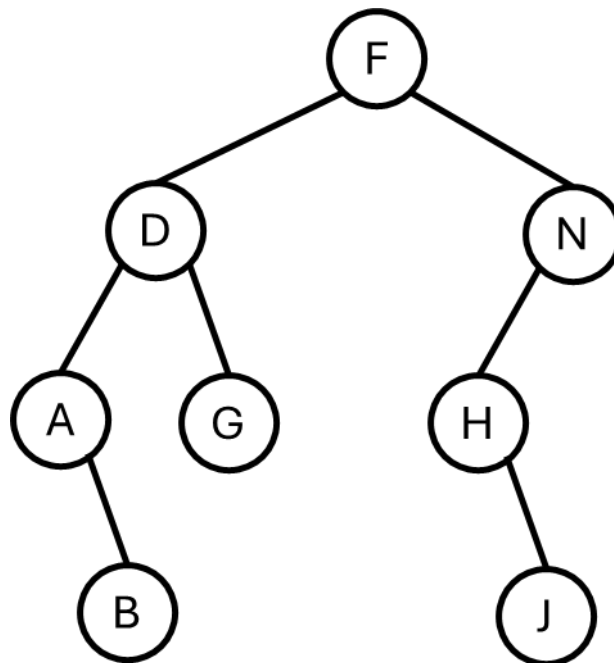
- i) Write the class definitions that correspond to the elements in the class diagram. The student's limit should be set according to the number of courses the student is attending.
- ii) Implement functionality for the `borrow_book()` methods.
- iii) Add docstrings and in-code comments in your code for parts i) and ii). These should follow the Documentation Requirements.

3. Consider the following list:

`["14:38", "09:12", "18:05", "06:00", "12:10", "17:01"]`

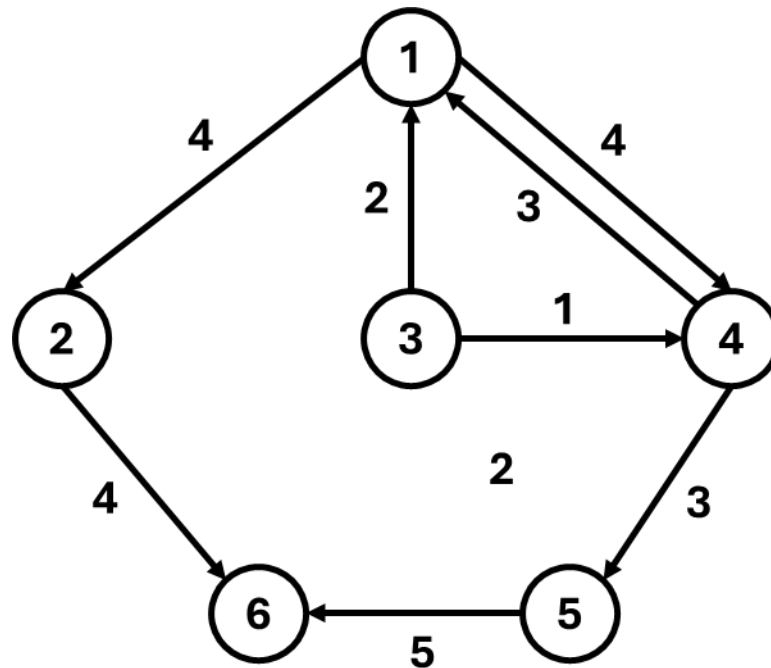
- i) By hand (without using code) sort the list of times using the heapsort algorithm in descending chronological order. Trace the execution of the algorithm in detail.
- ii) Write a class called `Schedule` that contains a heap instance variable (`Heap` class in Appendix A. Modify the code so that the elements are compared based on chronological order as in part i). This class should take a list as an argument of its constructor and store the data passed in as a heap. Instantiate this class with the list above.
- iv) Add docstrings and in-code comments in your code for part ii). These should follow the Documentation Requirements.

4. Consider the following tree diagram and the binary tree node class in Appendix B:



- i) By hand (without using code) perform preorder depth-first traversal of this tree. Clearly explain which nodes are visited and why.
- ii) Write code to create this tree object using the Node class. Write a comment stating whether this tree is an example of a binary search tree or not. Justify your answer.
- iii) Modify the way Search_tree class in Appendix C works. It should work as normal, but if the input is an uppercase character, it is inserted into the right subtree of the root, while if it is a lowercase character, it should be inserted into the left subtree of the root. If the input is invalid handle the error.
Note: use the function `is_str` shown in Appendix D to check if a value is a String.
- iv) Add docstrings and in-code comments in your code for parts ii) and iii) as appropriate. These should follow the Documentation Requirements.

5. Consider the following graph diagram and the Graph class in Appendix E:



- i) By hand (without using code) perform breadth-first traversal of this graph starting at Vertex 5. Clearly explain which nodes are visited and why.
- ii) Modify the Graph class in Appendix E to make a `DirectedWeightGraph`. Add a dunder method to the `DirectedWeightGraph` class that could be used to add two graphs together. Write code to create the graph above using the `DirectedWeightGraph` class you create.
- iii) Add a docstrings and in-code comments in your code for part ii) as appropriate. These should follow the Documentation Requirements.

Appendix A

```
class Heap:
    def __init__(self, data = []):
        self.data = data

    def is_empty(self):
        return len(self.data) == 0

    def parent(self, index):
        return (index - 1) // 2

    def left(self, index):
        return index * 2 + 1

    def right(self, index):
        return index * 2 + 2

    def insert(self, value):
        pos = len(self.data)
        self.data.append(0)
        while pos > 1 and self.data[self.parent(pos)] < value:
            self.data[pos] = self.data[self.parent(pos)]
            pos = self.parent(pos)
        self.data[pos] = value

    def extract(self):
        max_value = self.data[0]
        self.data[0] = self.data[len(self.data)-1]
        self.data.pop()
        self.move_down(0, len(self.data)-1)
        return max_value

    def move_down(self, first, last):
        while self.left(first) <= last:
            larger = self.left(first)
            if (self.right(first) <= last and
                self.data[self.left(first)] < self.data[self.right(first)]):
                larger = self.right(first)
            if self.data[first] >= self.data[larger]:
                break
            temp = self.data[first]
            self.data[first] = self.data[larger]
            self.data[larger] = temp
            first = larger
```

```
def heap_sort(data):
    heap = Heap(data)
    for i in range(len(heap.data)//2 - 1, -1, -1):
        heap.move_down(i, len(heap.data)-1)

    for i in range(len(heap.data) - 1, 0, -1):
        temp = heap.data[0]
        heap.data[0] = heap.data[i]
        heap.data[i] = temp
        heap.move_down(0, i-1)

    return heap
```

Appendix B

Binary Tree Node class:

```
class Node:
    def __init__(self, val, l = None, r = None):
        self.value = val
        self.left = l
        self.right = r

    def __str__(self):
        if self.left == None and self.right == None:
            return str(self.value)
        else:
            return str(self.value) + "({})({})".format(
                str(self.left) if self.left != None else "",
                str(self.right) if self.right != None else "")
```

Appendix C

```
from Node import Node

class Search_tree:
    def __init__(self):
        self.root = None

    def insert(self, val, node = "root"):
        if node == "root":
            self.root = self.insert(val, self.root)
            # If node is a empty, create a new node
        elif node == None:
            return Node(val)
            # Otherwise, recur through the tree
        else:
            # Insert into left subtree
            if val < node.value:
                node.left = self.insert(val, node.left)
            # Insert into right subtree
            elif val > node.value:
                node.right = self.insert(val, node.right)
            return node
```

Appendix D

The function to check if a value is an integer

```
def is_int(value):  
    return isinstance(value, int)
```

The function to check if a value is a string

```
def is_str(value):  
    return isinstance(value, str)
```

Appendix E

Simple graph class:

```
from collections import defaultdict
from Queue import Queue

class Graph:
    def __init__(self):
        self.vertices = set()
        self.edges = defaultdict(list)

    def add_vertex(self, v):
        self.vertices.add(v)

    def add_edge(self, v, u):
        self.edges[v].append(u)
        self.edges[u].append(v)
```